

Homework 06

Rahaf Alabdullah (rahafaa2)

Question 1: Compare KNN and logistic regression

```
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

data_dir = Path(__file__).parent / "data"
data = pd.read_csv(data_dir / "breast-cancer.csv")
predictors = ["mean_radius", "mean_texture", "mean_smoothness"]
X, y = data[predictors].to_numpy(), data["y"].to_numpy()
split = pd.read_csv(data_dir / "breast-cancer-split.csv")
train = split["split"] == "train"
test = split["split"] == "test"
X_train, X_test = X[train], X[test]
y_train, y_test = y[train], y[test]

mean_X = X_train.mean(axis=0)
sd = X_train.std(axis=0)
X_train_s = (X_train - mean_X) / sd
X_test_s = (X_test - mean_X) / sd

fit = LogisticRegression(C=np.inf, solver="lbfgs", tol=1e-10, max_iter=1000)
fit.fit(X_train_s, y_train)

k_values = np.array([5, 15, 25, 35, 45])
model_names = ["Logistic regression"] + [f"KNN, k = {k}" for k in k_values]
p_test = np.empty((len(y_test), len(model_names)))
p_test[:, 0] = fit.predict_proba(X_test_s)[:, 1] #store the results from logistic regression in the first column

for j, k in enumerate(k_values):
    knn_fit = KNeighborsClassifier(n_neighbors=k, weights="uniform", metric="euclidean")
    knn_fit.fit(X_train_s, y_train)
    p_test[:, j+1] = knn_fit.predict_proba(X_test_s)[:, 1]

auc_list = []
fig, ax = plt.subplots(figsize=(6.5, 6))
ax.plot([0, 1], [0, 1], color="0.7", linestyle="--")
for name, p in zip(model_names, p_test.T):
    fpr, tpr, _ = roc_curve(y_test, p)
    auc = roc_auc_score(y_test, p)
    auc_list.append(auc)
    ax.plot(fpr, tpr, label=f"{name} (AUC = {auc:.3f})")
ax.set(xlim=(0, 1), ylim=(0, 1),
       xlabel="False-positive rate", ylabel="True-positive rate")
ax.legend(loc="lower right", frameon=False, fontsize=9)
fig.tight_layout()
plt.show()
print(pd.DataFrame({"Model": model_names, "Test AUC": auc_list})
      .round(4).to_string(index=False))
```

Test AUC values

Model	Test AUC
Logistic regression	0.9746
KNN, $k = 5$	0.9329
KNN, $k = 15$	0.9480

Model	Test AUC
KNN, $k = 25$	0.9599
KNN, $k = 35$	0.9727
KNN, $k = 45$	0.9741

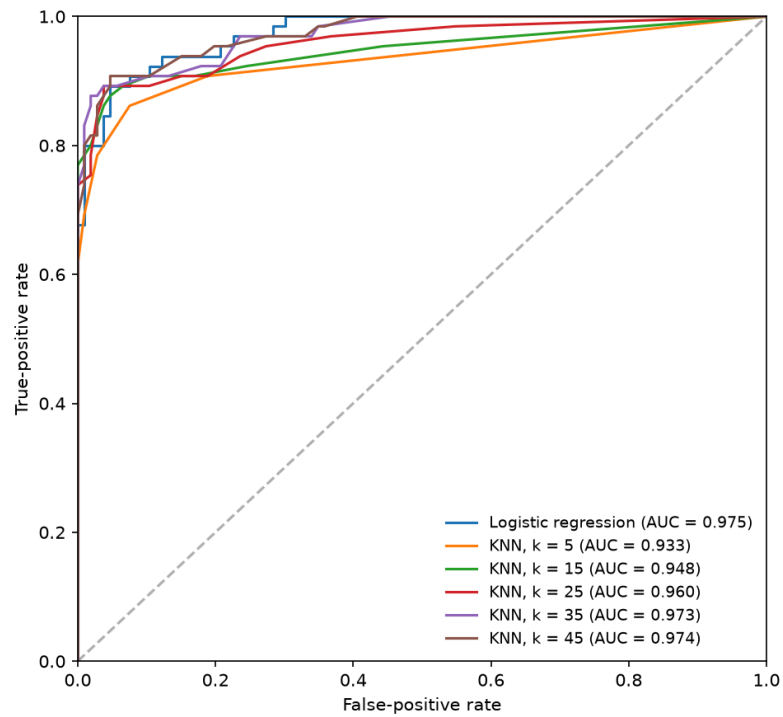


Figure 1: Test ROC curves for logistic regression and KNN

All of the models achieved similar AUC values, with logistic regression achieving the highest AUC. I would prefer logistic regression because it is more interpretable and has a higher AUC than the KNN models. As for the KNN models, the AUC increases with increasing k , which is expected because larger k values lead to smoother probability estimates but may also introduce bias.

Question 2: Training and test ROC curves

```
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_curve, roc_auc_score

data_dir = Path(__file__).parent / "data"
data = pd.read_csv(data_dir / "breast-cancer.csv")
predictors = ["mean_radius", "mean_texture", "mean_smoothness"]
X, y = data[predictors].to_numpy(), data["y"].to_numpy()

split = pd.read_csv(data_dir / "breast-cancer-split.csv")
train = split["split"] == "train"
test = split["split"] == "test"
X_train, X_test = X[train], X[test]
y_train, y_test = y[train], y[test]

mean_X = X_train.mean(axis=0)
sd = X_train.std(axis=0)
X_train_s = (X_train - mean_X) / sd
X_test_s = (X_test - mean_X) / sd

k_values = np.array([1, 3, 5, 10, 20])
model_names = [f"KNN, k = {k}" for k in k_values]
P_train = np.empty((len(y_train), len(k_values)))
p_test = np.empty((len(y_test), len(k_values)))
for j, k in enumerate(k_values):
    knn_fit = KNeighborsClassifier(n_neighbors=k, weights="uniform", metric="euclidean")
    knn_fit.fit(X_train_s, y_train)
    P_train[:, j] = knn_fit.predict_proba(X_train_s)[:, 1]
    p_test[:, j] = knn_fit.predict_proba(X_test_s)[:, 1]

auc_train = np.empty(len(k_values))
auc_test = np.empty(len(k_values))
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
axes = axes.ravel()
for m, name in enumerate(model_names):
    ax = axes[m]
    fpr_train, tpr_train, _ = roc_curve(y_train, P_train[:, m])
    fpr_test, tpr_test, _ = roc_curve(y_test, p_test[:, m])
    auc_train[m] = roc_auc_score(y_train, P_train[:, m])
    auc_test[m] = roc_auc_score(y_test, p_test[:, m])
    ax.plot(fpr_train, tpr_train, label=f"Train (AUC = {auc_train[m]:.3f})")
    ax.plot(fpr_test, tpr_test, label=f"Test (AUC = {auc_test[m]:.3f})")
    ax.plot([0, 1], [0, 1], color="0.7", linestyle="--")
    ax.set_title(f"k = {k_values[m]}")
    ax.set(xlim=(0, 1), ylim=(0, 1), xlabel="False-positive rate", ylabel="True-positive rate")
    ax.legend(loc="lower right", frameon=False, fontsize=9)
axes[-1].axis("off")
fig.tight_layout()
plt.show()

print(pd.DataFrame({"Model": model_names, "Train AUC": auc_train, "Test AUC": auc_test})
      .round(4).to_string(index=False))
```

Training and test AUC values

Model	Train AUC	Test AUC
KNN, $k = 1$	1.0000	0.8764
KNN, $k = 3$	0.9905	0.9138
KNN, $k = 5$	0.9847	0.9329
KNN, $k = 10$	0.9862	0.9446
KNN, $k = 20$	0.9880	0.9586

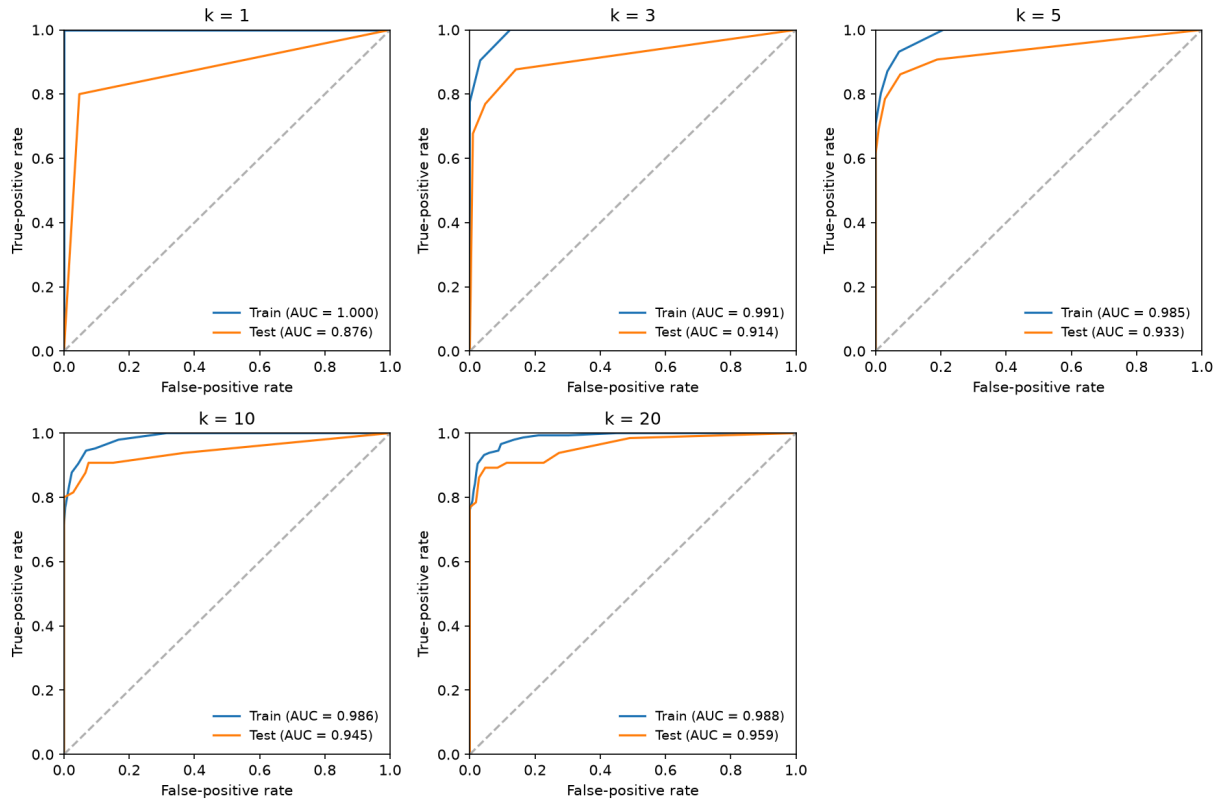


Figure 2: Training and test ROC curves for each k

The training AUC is higher than the test AUC for all k values. This is expected because the model has already seen the training data, leading to overfitting. For $k = 1$, the training AUC is 1.0, indicating perfect classification on the training set, but this does not generalize well to new observations. As k increases, the gap between train and test AUC decreases, suggesting that larger k values lead to smoother probability estimates and better generalization to unseen data.

Question 3: Peer evaluation of Homework 5

Review 1: Gicooaidun's report, Question 1 part (b)

Posted issue: <https://github.com/Gicooaidun/stat432-fall2026/issues/2>

1. Mistake / item for improvement. The table header row for k cuts off at 25. The values for $k = 27$ and $k = 29$ were clipped off due to column overflow in the output table.

2–3. Assessment and correction. To fix the table formatting issue in Question 1, Part (b), transpose the orientation so that the k values form rows rather than columns. Note: since the code isn't included in the report, I can't verify what you wrote, so I'll make my suggestion general:

```
import pandas as pd

# Assuming bias, sq_bias, variance, and mse are arrays or lists of length len(k_grid):
df_decomp = pd.DataFrame(
    {
        "k": k_grid,
        "Bias": bias,
        "Squared Bias": sq_bias,
        "Variance": variance,
        "MSE": mse,
    }
)

# Format floats to 4 decimal places:
print(df_decomp.to_string(index=False, float_format="%.4f"))
```

Review 2: Zhijing2023's report, Question 1 part (b)

Posted issue: <https://github.com/Zhijing2023/stat432-fall2026/issues/2>

1. Mistake / item for improvement. On Question 1, part (b), the stated statement is misleading. The text statement:

“Among the prediction matrix, each column contains 200 estimates for the same target. $f(x_0)$ measures the bias, and it's corresponding variance is the sampling variance. The average squared difference of $f(x_0)$ is the MSE.”

2. Assessment. There are two issues with this statement:

1. $f(x_0)$ does not measure the bias; the bias is the difference between the average prediction and the truth: $\text{Bias} = E[\hat{f}(x_0)] - f(x_0)$.
2. The MSE is the average squared difference between the estimator $\hat{f}(x_0)$ and the true value $f(x_0)$: $E[(\hat{f}(x_0) - f(x_0))^2]$, not differences of $f(x_0)$ itself.

Also, the plot was included without explanation.

3. Correction. An appropriate way to explain the result could be:

“As k increases, squared bias steadily increases while variance steadily decreases — averaging over more, and increasingly distant, neighbors smooths out the fitted mean (raising bias) while reducing sensitivity to the noise in any one training set (lowering variance). MSE is minimized at a small-to-moderate k , where the two effects balance. This mirrors the bias-variance trade-off in ridge and lasso, except that the trade-off there is controlled by the penalty strength λ rather than by neighborhood size: small λ (like small k) gives low bias and high variance, while large λ (like large k) gives high bias and low variance.”